

Document Technique — Système de Gestion des Hypothèques (SGH)

Société Ivoirienne de Banque (SIB)

Version : 1.0

Date : 18 août 2026

Auteur : Ibrahim Coulibaly — Accenture

Dépôt source : <https://github.com/Ibrahim044653/Garantie>

Table des matières

- [Contexte et objectifs](#)
- [Architecture générale](#)
- [Infrastructure et déploiement](#)
- [Structure du monorepo](#)
- [Modèle de données](#)
- [API — Description des endpoints](#)
- [Algorithme de calcul VNC](#)
- [Sécurité](#)
- [Contrôle d'accès basé sur les rôles \(RBAC\)](#)
- [Flux métier principaux](#)
- [Points d'attention et recommandations](#)

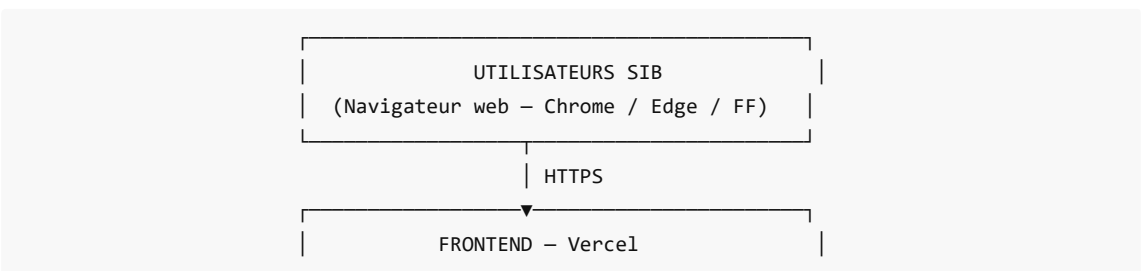
1. Contexte et objectifs

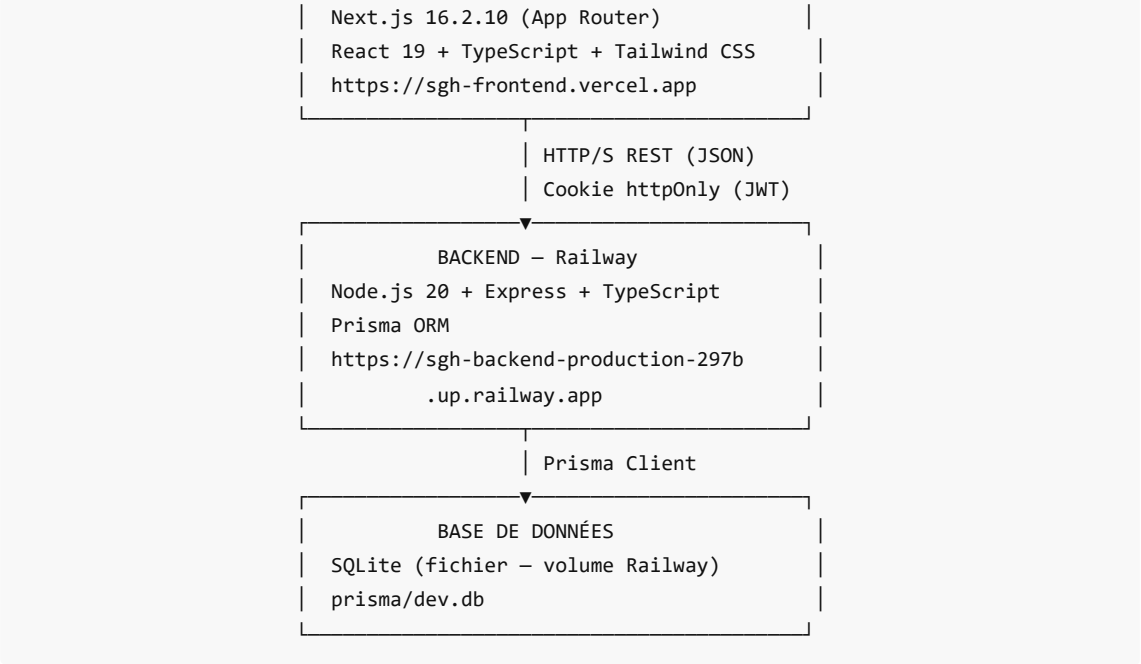
Le SGH est une application web interne développée pour la SIB afin de centraliser et fiabiliser la gestion du portefeuille des garanties hypothécaires. Avant cet outil, le suivi des hypothèques reposait sur des fichiers Excel disparates, ce qui exposait la banque à des risques opérationnels (erreurs de calcul, péremptions non détectées, absence de traçabilité).

Objectifs fonctionnels :

- Enregistrement et suivi du cycle de vie de chaque hypothèque (bien immobilier, client, prêt associé)
- Calcul automatique de la Valeur Nette de Garantie (VNC) selon la circulaire BCEAO n°04-2017
- Génération d'alertes de péremption et de dépassement de seuil LTV
- Reporting annuel et exports Excel à destination des équipes Risques et Audit
- Authentification sécurisée avec MFA (TOTP) et gestion des droits par rôle

2. Architecture générale





Flux de communication :

- 1. Le navigateur charge l'application Next.js depuis Vercel (SSR/CSR selon la route).
- 2. Toutes les requêtes API transitent vers le backend Railway via HTTPS.
- 3. Le jeton JWT est stocké en cookie httpOnly (non accessible au JavaScript de la page), ce qui prévient le vol par XSS.
- 4. Les réponses API sont en JSON ; le module `lib/api.ts` (Axios) gère les intercepteurs de token et les erreurs 401.

3. Infrastructure et déploiement

3.1 Frontend — Vercel

Paramètre	Valeur
Framework	Next.js 16.2.10 (App Router)
Hébergement	Vercel (CDN mondial)
URL de production	https://sgh-frontend.vercel.app
Build	<code>next build</code> (output statique + SSR edge)
Variables d'env	<code>NEXT_PUBLIC_API_URL</code> → URL du backend Railway

3.2 Backend — Railway

Paramètre	Valeur
Runtime	Node.js 20 LTS
Framework	Express 4.x + TypeScript

ORM	Prisma 5.x
Base de données	SQLite (fichier persisté sur volume Railway)
URL de production	https://sgh-backend-production-297b.up.railway.app
Fichier de config	railway.toml (à la racine du monorepo)
Build command	npm run build --workspace=backend
Start command	node backend/dist/index.js

Le fichier `railway.toml` à la racine orchestre le build depuis le monorepo : Railway exécute `prisma generate` puis `prisma db push` avant de démarrer le serveur, ce qui garantit que le schéma SQLite est toujours synchronisé avec le code déployé.

4. Structure du monorepo

```
Hypothèque/
├─ package.json          # Workspaces root (Railway build entry)
├─ railway.toml          # Configuration déploiement Railway
├─
├─ frontend/            # Application Next.js
│   └─ app/
│       └─ login/        # Page d'authentification (JWT + MFA TOTP)
│           └─ (dashboard)/ # Groupe de routes protégées (layout commun)
│               └─ hypothèques/ # Liste, fiche détail, réévaluation
│                   └─ alertes/ # Tableau des alertes, marquage lu
│                       └─ reporting/ # Reporting annuel, exports PDF/Excel
│                           └─ admin/users/ # Gestion des utilisateurs (ADMIN seulement)
│                               └─ profil/ # Profil utilisateur, activation MFA
│   └─ components/      # Composants React réutilisables
│   └─ lib/
│       └─ api.ts        # Instance Axios avec intercepteurs JWT
│       └─ types/        # Types TypeScript partagés frontend
│   └─
└─ backend/
    └─ src/
        └─ index.ts      # Point d'entrée Express, configuration CORS
        └─ controllers/
            └─ auth.controller.ts
            └─ hypothèque.controller.ts
            └─ dashboard.controller.ts
            └─ reporting.controller.ts
            └─ alerte.controller.ts
            └─ mfa.controller.ts
            └─ user.controller.ts
        └─ routes/      # Fichiers de routage Express par domaine
        └─ services/
            └─ calcul.service.ts # Moteur de calcul VNC (cf. §7)
            └─ alert.service.ts  # Génération automatique des alertes
```

```
|   └─ middleware/
|       └─ auth.middleware.ts # Vérification JWT + contrôle RBAC
└─ prisma/
    ├── schema.prisma        # Définition du schéma de base de données
    └─ seed.ts                # Données initiales (utilisateurs, rôles)
```

5. Modèle de données

Le schéma Prisma définit quatre entités principales.

5.1 User

Champ	Type Prisma	Description
id	Int (PK, autoincrement)	Identifiant interne
email	String (unique)	Identifiant de connexion
password	String	Hash bcrypt (coût 10)
nom	String	Nom de famille
prenom	String	Prénom
role	Enum Role	Rôle RBAC (cf. §9)
mfaSecret	String?	Clé secrète TOTP (chiffrée)
mfaEnabled	Boolean	Indicateur MFA actif

5.2 Hypotheque

Champ	Type Prisma	Description
id	Int (PK)	Identifiant interne
codeClient	String	Code client SIB
nomClient	String	Nom complet du client
numeroPret	String	Référence du prêt associé
numeroTitreFoncier	String	Numéro de titre foncier officiel
natureBien	String	Ex. : Villa, Appartement, Terrain
ville	String	Ville de localisation du bien
quartier	String	Quartier
lot	String?	Numéro de lot
ilot	String?	Numéro d'ilot
zoneGeographique	Enum Zone	A, B, C ou INDUSTRIELLE

statutOccupation	Enum Occupation	LIBRE, OCCUPE_PROPRIETAIRE, LOUE_AVEC_BAIL
valeurExpertiseInitiale	Float	Valeur expertisée à l'origine (XOF)
dateExpertise	DateTime	Date du rapport d'expertise
montantInscription	Float	Montant inscrit au livre foncier (XOF)
rangHypothèque	Int	Rang (1er, 2ème...)
datePeremptionInscription	DateTime	Date d'expiration de l'inscription
soldePret	Float	Encours restant dû (XOF)
dateEcheancePret	DateTime	Échéance finale du prêt
pjExpertisePath	String?	Chemin du fichier rapport d'expertise

5.3 HistoriqueValeur

Enregistre chaque réévaluation ou revalorisation de la garantie.

Champ	Type Prisma	Description
id	Int (PK)	Identifiant
hypothèqueId	Int (FK)	Référence vers Hypothèque
valeurExpertise	Float	Valeur expertisée retenue
decoteZone	Float	Décote géographique appliquée (%)
decoteAnciennete	Float	Décote d'ancienneté appliquée (%)
decoteOccupation	Float	Décote d'occupation appliquée (%)
decoteTotale	Float	Décote totale (plafonnée à 100 %)
vnc	Float	Valeur Nette de Garantie calculée (XOF)
loanToValue	Float	Ratio LTV en %
modifiePar	String	Email de l'auteur de la modification
motif	String	Justification de la réévaluation
dateModification	DateTime	Horodatage

5.4 Alert

Champ	Type Prisma	Description
id	Int (PK)	Identifiant
hypothèqueId	Int (FK)	Référence vers Hypothèque
type	String	Ex. : PEREMPTION, LTV_DEPASSE, ECHEANCE

message	String	Libellé de l'alerte
dateEcheance	DateTime?	Date déclenchante
lu	Boolean	Statut de lecture

6. API — Description des endpoints

Toutes les routes (sauf `/api/auth/login`) nécessitent un JWT valide transmis soit en cookie `httpOnly token` , soit en en-tête `Authorization: Bearer <token>` .

6.1 Authentification

Méthode	Route	Description
POST	<code>/api/auth/login</code>	Connexion. Retourne <code>{ token }</code> ou <code>{ mfaRequired: true }</code> si MFA activé
GET	<code>/api/auth/mfa/setup</code>	Génère le secret TOTP et l'URI QR code
POST	<code>/api/auth/mfa/confirm</code>	Valide le premier code TOTP et active le MFA
POST	<code>/api/auth/mfa/validate</code>	Valide un code TOTP lors d'une connexion avec MFA

6.2 Hypothèques

Méthode	Route	Rôles autorisés	Description
GET	<code>/api/hypotheques</code>	Tous	Liste paginée avec filtres
POST	<code>/api/hypotheques</code>	ADMIN, GESTIONNAIRE_GARANTIES	Création d'une hypothèque
GET	<code>/api/hypotheques/:id</code>	Tous	Détail d'une hypothèque
PUT	<code>/api/hypotheques/:id</code>	ADMIN, GESTIONNAIRE_GARANTIES	Mise à jour
DELETE	<code>/api/hypotheques/:id</code>	ADMIN	Suppression
POST	<code>/api/hypotheques/:id/reevaluer</code>	ADMIN, GESTIONNAIRE_GARANTIES	Réévaluation manuelle (nouveau rapport d'expertise + motif)
POST	<code>/api/hypotheques/:id/revaloriser</code>	ADMIN, GESTIONNAIRE_GARANTIES	Revalorisation par indice (<code>indiceRevalorisation</code> en %)
GET	<code>/api/hypotheques/:id/historique</code>	Tous sauf ENGAGEMENTS	Historique des valorisations

6.3 Dashboard

Méthode	Route	Description
---------	-------	-------------

GET	/api/dashboard/stats	Statistiques globales : nombre de dossiers, VNC totale, alertes actives, répartition par zone
-----	----------------------	---

6.4 Reporting

Méthode	Route	Description
GET	/api/reporting/annuel	Rapport de synthèse annuel (JSON)
GET	/api/reporting/export	Export PDF du rapport annuel
GET	/api/reporting/export-excel	Export XLSX avec détail de chaque hypothèque

6.5 Alertes

Méthode	Route	Description
GET	/api/alertes	Liste des alertes actives (non lues en priorité)
PUT	/api/alertes/:id/lu	Marquer une alerte comme lue
PUT	/api/alertes/marquer-tout-lu	Marquer toutes les alertes comme lues

6.6 Utilisateurs (ADMIN)

Méthode	Route	Description
GET	/api/users	Liste des utilisateurs
POST	/api/users	Création d'un utilisateur
PUT	/api/users/:id	Modification (rôle, informations)
DELETE	/api/users/:id	Suppression

7. Algorithme de calcul VNC

Le moteur de calcul est encapsulé dans `backend/src/services/calcul.service.ts`. Il implémente la méthodologie de valorisation des garanties immobilières définie par la circulaire BCEAO n°04-2017.

7.1 Paramètres d'entrée

- `valeurExpertise` : valeur issue du dernier rapport d'expertise (XOF)
- `dateExpertise` : date du rapport (pour le calcul de l'ancienneté)
- `zoneGeographique` : enum A | B | C | INDUSTRIELLE
- `statutOccupation` : enum LIBRE | OCCUPE_PROPRIETAIRE | LOUE_AVEC_BAIL
- `soldePret` : encours restant dû (XOF)

7.2 Barèmes des décotes

Décote de zone géographique (`D_zone`) :

Zone	Décote
------	--------

A (Centre-ville, ZAC)	20 %
B (Périphérie structurée)	30 %
C (Zone rurale / périurbaine)	45 %
INDUSTRIELLE	40 %

Décote d'ancienneté de l'expertise (D_anc) :

L'ancienneté est calculée à partir de la date d'expertise jusqu'à la date du calcul.

Ancienneté	Décote
0 à 3 ans	0 %
3 à 5 ans	10 %
Plus de 5 ans	100 % (expertise obsolète — bien non valorisable)

Décote d'occupation (D_occ) :

Statut d'occupation	Décote
LIBRE	0 %
OCCUPE_PROPRIETAIRE	5 %
LOUE_AVEC_BAIL	15 %

7.3 Formules de calcul

```
decoteTotale = Math.min(D_zone + D_anc + D_occ, 100)
```

```
VNC = valeurExpertise × (1 - decoteTotale / 100)
```

```
LTV (Loan-to-Value) = soldePret / VNC × 100
```

La décote totale est plafonnée à 100 % : si la somme des trois décotes dépasse 100 %, la VNC est nulle et le bien ne peut pas servir de garantie recevable.

7.4 Enregistrement dans l'historique

Chaque exécution du calcul (déclenchée par une réévaluation ou une revalorisation) génère un enregistrement `HistoriqueValeur` avec le détail de chaque décote, la VNC calculée, le LTV, l'auteur et le motif. Ce registre est immuable : aucun enregistrement passé ne peut être modifié ou supprimé, ce qui garantit la traçabilité exigée par l'audit interne.

7.5 Revalorisation par indice

La revalorisation (endpoint `POST /api/hypotheques/:id/revaloriser`) applique un indice de marché à la valeur d'expertise courante sans nécessiter de nouveau rapport physique :

```
nouvelleValeurExpertise = valeurExpertiseActuelle × (1 + indiceRevalorisation / 100)
```

Le calcul VNC est ensuite relancé avec cette nouvelle valeur.

8. Sécurité

8.1 Authentification JWT

Le backend émet un JSON Web Token (JWT) signé avec une clé secrète stockée en variable d'environnement (`JWT_SECRET`). Le token est transmis au navigateur via un **cookie httpOnly** (inaccessible au JavaScript de la page, ce qui prévient le vol par attaque XSS). Pour les appels API directs (Postman, scripts), le token peut également être transmis en en-tête `Authorization: Bearer <token>` .

Le middleware `auth.middleware.ts` vérifie la signature et l'expiration du token sur chaque requête protégée.

8.2 Hachage des mots de passe — `bcryptjs`

Les mots de passe utilisateurs ne sont jamais stockés en clair. Lors de la création ou de la modification du mot de passe, `bcryptjs` applique un hachage avec un coût de dérivation de 10 (par défaut), ce qui rend la compromission par force brute coûteuse en calcul.

```
// Exemple de hachage à la création d'un utilisateur
const hash = await bcrypt.hash(plainPassword, 10);
```

8.3 MFA — TOTP via `speakeasy`

L'authentification à deux facteurs est implémentée avec le protocole TOTP (Time-based One-Time Password, RFC 6238) via la bibliothèque `speakeasy` .

Flux d'activation (une seule fois, depuis la page Profil) :

1. GET `/api/auth/mfa/setup` : le backend génère un secret TOTP et retourne une URI compatible avec les applications d'authentification (Google Authenticator, Authy...). Le frontend affiche le QR code correspondant.
2. L'utilisateur scanne le QR code et saisit le premier code à 6 chiffres.
3. POST `/api/auth/mfa/confirm` : le backend vérifie le code et, s'il est valide, enregistre le secret et positionne `mfaEnabled = true` .

Flux de connexion avec MFA actif :

1. POST `/api/auth/login` avec email + mot de passe → le backend retourne `{ mfaRequired: true }` sans émettre de JWT.
2. Le frontend redirige vers l'écran de saisie du code TOTP.
3. POST `/api/auth/mfa/validate` avec le code à 6 chiffres → le backend émet le JWT si le code est valide.

8.4 CORS

Le backend configure Express-CORS pour n'accepter les requêtes que depuis l'origine Vercel de production (`https://sgh-frontend.vercel.app`) et, en environnement de développement, depuis `http://localhost:3000` .

8.5 Variables d'environnement sensibles

Les secrets (clé JWT, base de données, etc.) ne sont jamais commités dans le dépôt Git. Ils sont injectés via les interfaces de secrets de Vercel (frontend) et Railway (backend).

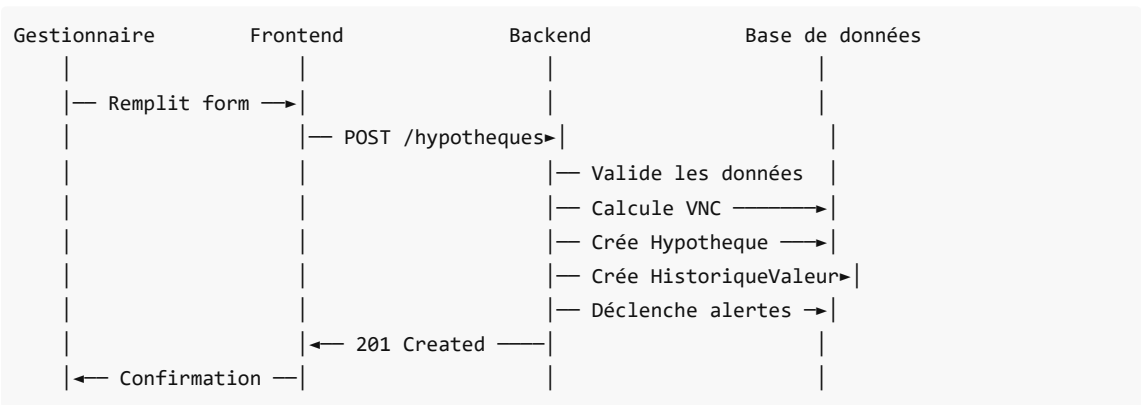
9. Contrôle d'accès basé sur les rôles (RBAC)

Le middleware `auth.middleware.ts` vérifie le rôle de l'utilisateur authentifié avant d'autoriser l'accès à chaque route. Le rôle est encodé dans le payload JWT et relu à chaque requête.

Rôle	Hypothèques	Réévaluation	Export	Historique	Administration users
ADMIN	CRUD complet	Oui	Oui	Oui	Oui
GESTIONNAIRE_GARANTIES	CRUD complet	Oui	Oui	Oui	Non
RESPONSABLE_RISQUES	Lecture seule	Non	Oui	Oui	Non
ENGAGEMENTS	Lecture seule	Non	Non	Non	Non
AUDIT_INTERNE	Lecture seule	Non	Oui	Oui (complet)	Non

10. Flux métier principaux

10.1 Enregistrement d'une nouvelle hypothèque



10.2 Réévaluation d'une hypothèque

La réévaluation est déclenchée lorsqu'un nouveau rapport d'expertise est disponible (par exemple après 3 ans). Le gestionnaire saisit la nouvelle valeur d'expertise et un motif obligatoire.

```
POST /api/hypotheques/:id/reevaluer
Body: { valeurExpertise, dateExpertise, motif }
```

→ calcul.service recalcule VNC et LTV
→ HistoriqueValeur créé avec toutes les décotes détaillées
→ alert.service vérifie si LTV > seuil configuré et crée une alerte si nécessaire
→ Hypotheque.valeurExpertiseInitiale mise à jour

10.3 Génération des alertes

Le service `alert.service.ts` est appelé à chaque modification d'hypothèque et peut être exécuté en tâche planifiée (cron). Il génère des alertes de type :

- **PEREMPTION** : `datePeremptionInscription` à moins de 90 jours
- **ECHEANCE** : `dateEcheancePret` à moins de 60 jours
- **LTV_DEPASSE** : LTV calculé dépasse 80 % (seuil paramétrable)
- **EXPERTISE_OBSOLETE** : ancienneté de l'expertise supérieure à 3 ans

10.4 Export Excel

```
GET /api/reporting/export-excel
```

- Récupère toutes les hypothèques avec leur dernière VNC
- Génère un fichier XLSX (via bibliothèque `exceljs` ou `xlsx`)
- Retourne le fichier en pièce jointe (`Content-Disposition: attachment`)

11. Points d'attention et recommandations

11.1 Base de données SQLite en production

SQLite est adapté pour un MVP ou une utilisation à faible charge concurrente. Pour une montée en charge ou un déploiement multi-instances, une migration vers **PostgreSQL** (nativement supporté par Prisma et Railway) est recommandée. Le schéma Prisma ne nécessiterait que le changement du provider (`sqlite` → `postgresql`) et des migrations.

11.2 Persistance des fichiers PJ

Les pièces jointes (rapports d'expertise) sont actuellement stockées sur le système de fichiers du conteneur Railway (`pjExpertisePath`). Ce stockage est volatile entre les redéploiements. Il est recommandé de migrer vers un stockage objet externe (AWS S3, Cloudflare R2 ou Supabase Storage) avec signature d'URL temporaires pour le téléchargement sécurisé.

11.3 Alertes en temps réel

Le mécanisme d'alertes actuel est basé sur la vérification synchrone à chaque modification. Pour les alertes de péremption (basées sur des dates), il est recommandé d'ajouter un job cron (ex. `node-cron` ou Railway Cron Jobs) qui s'exécute quotidiennement et génère les alertes proactives pour l'ensemble du portefeuille.

11.4 Rotation des secrets JWT

La clé `JWT_SECRET` doit être rotée périodiquement. Prévoir un mécanisme de révocation des sessions actives (liste noire de JTI ou raccourcissement de la durée de vie du token avec refresh token).

11.5 Journalisation des accès sensibles

Pour répondre aux exigences de l'audit interne, il est recommandé d'ajouter un journal des accès aux fonctions sensibles (exports, suppressions, modifications de rôles) avec horodatage, identité de l'utilisateur et adresse IP source.